

Process Specifications and Structured Decisions

CSE 4407

Md. Bakhtiar Hasan

Assistant Professor
Department of Computer Science and Engineering
Islamic University of Technology

July 30, 2025



Process Specs: The “How-To” Guide for Your System’s Processes

- What
 - Detailed descriptions of a process’s logic
 - Also called “Mini Specs” → Small, focused parts of the total design
- When to Create
 - Created for *primitive* processes on a DFD
 - Sometimes for higher-level processes, if complex
 - Used for class methods and use case steps
- Goal: Explains the decision-making logic and formulae transforming inputs to outputs

Why Write Specs? The Goals of Process Specs

- **Reduce Ambiguity**
 - Forces detailed understanding
 - Uncovers vague areas for clarification
- **Provide Implementation Blueprint:** Clear instructions for programmers
- **Validate the System Design**
 - Ensures process has needed inputs
 - Confirms all inputs/outputs match the DFD

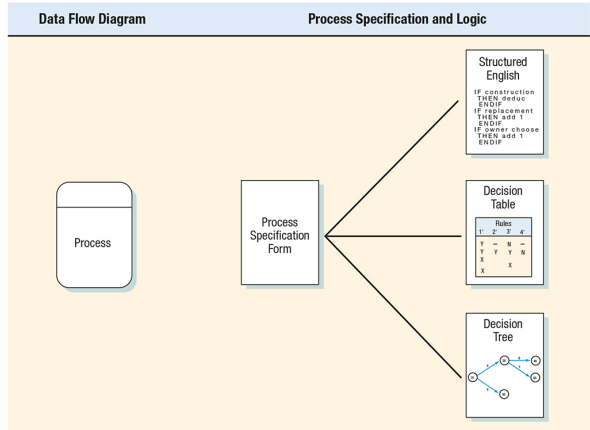
Exceptions: When Process Specs Aren't Needed

- Simple Physical I/O
 - Basics such as 'Read Customer Record' or 'Write Invoice Line'
 - Trivial logic
- Simple Data Validation
 - E.g., Checking if a field is numeric or within a range
 - Often handled by data dictionary rules or built-in GUI controls
 - *(Complex validation might still need a spec!)*
- Using Prewritten Code (Functions/Methods/Libraries)
 - Leveraging existing, documented code blocks
 - Why reinvent the wheel?

The Process Spec Format: Linking Everything Together

- Connects

- The Process on the DFD
- The Data Dictionary
- The Detailed Logic



© 2024 Pearson Education, Inc.

What Goes IN a Process Spec?

- Process Number
- Process Name
- **Brief Description:** What does it *do* in plain English?
- Input Data Flows
- Output Data Flows
- **Type of Process:** Batch? Online? Manual?
- **Prewritten Code:** If using existing code, name the subprogram, function, or method
- **Process Logic/Business Rules**
 - The CORE of the spec!
 - States policy/rules in everyday language, NOT code
 - Examples: Definitions, Conditions/Actions, Constraints, Formulae, Inferences, Sequences
- **Logic Reference:** If logic is complex, note its name/location
- **Unresolved Issues**
 - A list of questions, ambiguities, or things needing confirmation
 - Your 'to-do' list for follow-up!

Demo: [Process Specification Format Example](#)

Describing Logic: Structured English

- **Analogy:** Business Pseudocode → Less formal than code, more precise than plain English
- **What:** A way to write process logic using
 - **Structured Logic:** Sequences, decisions, and loops
 - **Simple English:** Common words
- **When to Use**
 - Processes involving formulae or calculations
 - Processes with looping/iteration
 - When structured decision are *not* overly complex

The Logic Structures

- **Sequential Structure:** Step-by-step instructions

Action #1

Action #2

Action #3

- **Decision Structure:** Choosing a path based on a condition

IF Condition A is True

 THEN implement Action A

ELSE implement Action B

ENDIF

The Logic Structures (cont.)

- **Case Structure:** A special decision for mutually exclusive choices

```
IF Case #1
```

```
    Then Implement Action #1
```

```
ELSE IF Case #2
```

```
    Then Implement Action #2
```

```
ELSE IF Case #3
```

```
    Then Implement Action #3
```

```
ELSE print error
```

```
ENDIF
```

- **Iteration Structure:** Repeating a block of actions

```
DO WHILE there are customers
```

```
    Action #1
```

```
ENDDO
```

Writing It Right: Structured English Conventions

- **Keyword Highlighting:** Capitalize standard logic words (IF, THEN, ELSE, DO, DO WHILE, DO UNTIL, PERFORM)
- **Indentation:** Indent blocks of statements within IFs, Cases, or DOs to show structure clearly
- **Data Dictionary Terms:** Underline words or phrases defined in the DD
- **Logic Clarity**
 - Be precise with AND/OR
 - Be clear about comparisons ($>$, $>=$, $<$, $<=$)
 - Eliminate ambiguity *before* coding

Example: Medical Claim Processing

User Description

We process all our claims in this manner. First, we determine whether the claimant has ever sent in a claim before; if not, we set up a new record. The claim totals for the year are then updated. Next, we determine if a claimant has policy A or policy B, which differ in deductibles and copayments (the percentage of claim claimants pay themselves). For both policies, we check to see if the deductible has been met (\$100 for policy A and \$50 for policy B). If the deductible has not been met, we apply the claim to the deductible. Another step adjusts for the copayment; we subtract the percentage the claimant pays (40 percent for policy A and 60 percent for policy B) from the claim. Then we issue a check if there is money coming to the claimant, print a summary of the transaction, and update our accounts. We do this until all claims for that day are processed.

- Identify sequences, decisions, cases, iterations, and potential ambiguities

Demo: [Medical Claim Processing Structured English](#)

Benefits of Structured English

- **Clarity**
 - Forces clear thinking about logic and relationships
 - Helps uncover and resolve ambiguities early
- **Communication**
 - Acts as a bridge between users and developers
 - Can be taught to and understood by non-technical users (with a little guidance)
 - Easier to review and validate business logic compared to code
- **Foundation for Code:** Provides a solid, logic-checked base for programmers

Data Dictionary: The Blueprint for Specs

- **Sequence of Fields:** Implies a simple sequence of actions (e.g., Move, Assign)
- **Optional Elements () or Selection []:** Implies an IF . . . THEN . . . ELSE structure
- **Iteration { }:** Implies a DO WHILE or DO UNTIL loop

Demo: [Shipping Statement Data Structure](#) vs. [Shipping Statement Structured English](#)

Handling Complex Choices: Decision Tables

- **What:** A structured table for representing complex decision logic
- **When to Use**
 - When Structured English becomes too nested or hard to follow due to many conditions
 - Excellent for visualizing complex combinations of conditions and resulting actions
 - Helps ensure all possible scenarios are considered

Anatomy of a Decision Table

Conditions and Actions	Rules
Conditions	Condition Alternatives
Actions	Action Entries

- **Conditions:** The factors influencing the decision
- **Condition Alternatives:** Possible values/combinations for conditions
- **Actions:** The possible outcomes or steps to take
- **Action Entries:** Which action(s) to take for each rule
- **Logic Flow:** Identify conditions, find the matching rule column, read down to find the action(s) marked with 'X'

Example Decision Table: Store Purchase Policy

Conditions and Actions	Rules			
	1	2	3	4
Under \$50	Y	Y	N	N
Pays by check with two forms of ID	Y	N	Y	N
Uses credit card	N	Y	N	Y
Complete the sale after verifying signature.	X			
Complete the sale. No signature needed.		X		
Call supervisor for approval.			X	
Communicate electronically with bank for credit card authorization.				X

- **Reading Example:** IF Sale NOT under \$50 AND Pays by check AND NOT Uses credit card, THEN Call Supervisor

How to Build a Decision Table

1. Determine Conditions

- What factors affect the decision?
- Combine overlapping/mutually exclusive conditions if possible

2. Determine Actions: What are all possible outcome/results?

3. Determine Condition Alternatives

- For each condition, what are the possible values? (e.g., Y/N, <18/18-65/>65, Red/Green/Blue)
- **Crucial:** Include ALL possibilities

4. Calculate Max Number of Rules (Columns): Multiple the number of alternatives for each condition (e.g., 3 Y/N conditions, $2 \times 2 \times 2 = 8$)

5. Fill in Condition Alternatives: Systematically list all possible combinations

How to Build a Decision Table (cont.)

6. **Complete Action Entries:** For each rule, mark the corresponding row of the correct action(s) in lower-right quadrant with 'X'
7. **Combine Rules**
 - Look for columns where the actions are the SAME, and only *some* conditions differ
 - If a condition's alternative does not affect the outcome for a set of rules leading to the same action, replace the values with a dash (Don't Care)
8. **Check for Errors:** Incompleteness, impossible situations, contradictions, and redundancy (Discussed later)
9. **Rearrange for Clarity (Optional):** Reorder conditions, actions, or even rules if it makes the table easier to understand

Simplifying Decision Table Example: Catalog Mailing List

Conditions and Actions	Rules							
	1	2	3	4	5	6	7	8
Customer ordered from fall catalog.	Y	Y	Y	Y	N	N	N	N
Customer ordered from holiday catalog.	Y	Y	N	N	Y	Y	N	N
Customer ordered from specialty catalog.	Y	N	Y	N	Y	N	Y	N
Send out this year's holiday catalog.		X		X		X		X
Send out specialty catalog.			X				X	
Send out both catalogs.	X				X			

Conditions and Actions	Rules		
	1' (1, 5)	2' (2, 4, 6, 8)	3' (3, 7)
Customer ordered from fall catalog.	-	-	-
Customer ordered from holiday catalog.	Y	-	N
Customer ordered from specialty catalog.	Y	N	Y
Send out this year's holiday catalog.		X	
Send out specialty catalog.			X
Send out both catalogs.	X		

Checking Your Table: Common Pitfalls

- Incompleteness

- Missing conditions? (e.g., Forgot to check if total order > \$50?)
- Missing alternatives? (e.g., Handled Red and Blue, forgot Green?)
- Missing actions? (e.g., What happens if ALL conditions are No?)

- Impossible Situations

- Rules and defy logic or reality (e.g., Salary > \$50,000/year = Y AND Salary < \$2,000/month = Y)
- Investigate why it appeared, likely remove/correct the rule

- Contradictions: Two (or more) rules have the **exact same condition** alternatives, but specify **different action(s)**

- Redundancy: Two (or more) rules have the **exact same condition** and also specify the **exact same action(s)**

Decision Tables: The Payoff

- **Completeness:** Systematic structure helps ensure all combinations are considered
- **Error Detection:** Makes it easier to spot impossible situations, contradictions, and redundancies
- **Handles Complexity:** Manages intricate logic with many interacting conditions more clearly than nested Structured English
- **Code Generation:** Can sometimes be used directly by “decision table processors” to automatically generate program code
- **Communication:** Can be reviewed with users (even though sometimes less intuitive than Structured English)

Visualizing Choices: Decision Trees

- What
 - A graphical representation of decision logic
 - Uses a tree structure → Usually drawn sideways
 - Shows branching sequences of conditions and actions
- When to Use
 - When decision logic involves complex branching
 - When the sequence of decisions is important
 - As an alternative to potentially large or complex decision tables

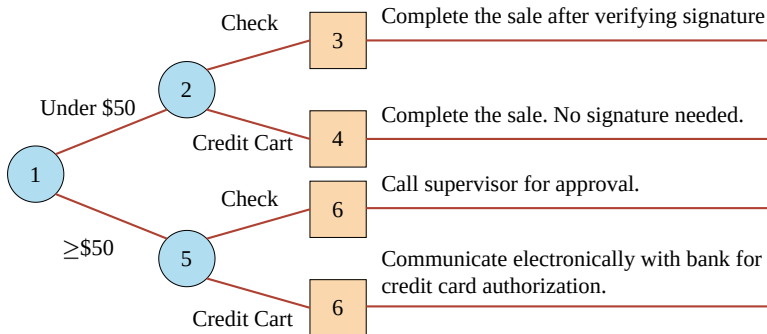
Anatomy of a Decision Tree

- **Structure**
 - **Root:** Starting point of the decision (on the left)
 - **Branches:** Represents different conditions or alternatives leading from a node
 - **Node:** Represent points where a condition is checked or an action occurs
- **Notation:** Numbered node for easy reference
 - **Circle Node:** Represents a Condition (an “IF” point)
 - **Square Node:** Represents an Action (a “THEN” outcome)

Planting Your Decision Tree: The Steps

- Identify Conditions, Actions, and Order
 - List all conditions to be checked
 - List all possible resulting actions
 - Determine the sequence in which conditions need to be checked → Crucial if timing matters
- Build Left-to-Right
 - Start with the initial condition as the root node (circle) on the left
 - Draw branches for *all possible alternatives* leading from that node
 - For each branch:
 - If it leads to another condition, draw a circle
 - If it leads to an action, draw a square
 - Continue branching until all paths end in an action node

Example Decision Tree: Store Purchase Policy



- **Reading a Path:** Start at Root, if the bill is more than \$50, and then customer pays with check, then call supervisor for approval

Decision Tree Characteristics

- **Sequence Matters:** Order of conditions is clear from left-to-right structure
- **Asymmetry is Normal**
 - Not all branches need to go to the same depth
 - Some paths might involve more condition checks than others
- **Relevance is Key:** Conditions and actions only appear on the branches where they are relevant
- **Action Can Repeat:** The same action might appear at the end of multiple different paths

Why Choose a Tree? Advantages Over Tables

- **Sequence Clarity:** Immediately shows the order of checking conditions and executing actions
- **Focus on Relevance**
 - Conditions/actions only appear on branches where they apply
 - Can feel less cluttered than tables for certain types of logic (especially asymmetrical)
- **Communication and Understanding**
 - Often more readily understood by non-technical stakeholders
 - The visual path-tracing can be more than intuitive than reading table columns

The Right Tool for the Job

- Three techniques
 - **Structured English:** Precise procedural description
 - **Decision Tables:** Handles complex combinations and aids error checking
 - **Decision Trees:** Visualizes sequence and branching logic
- **All three?** Usually NO, choose the *best* fit for the specific process logic you are documenting

Making the Choice

- **Structured English**

- The logic involves many repetitious actions → Handles loops naturally and concisely
- Communication to end users is paramount
 - Uses simple English statements
 - Can be easier for non-technical users to follow for sequential or moderately complex logic compared to tables/trees initially

- **Decision Tables**

- Facing complex combinations of conditions, actions, and rules → Tables excel at systematically organizing many interacting conditions
- Rigorous error checking is crucial → The systematic structure helps you effectively identify and avoid impossible situations, redundancies, and contradictions

- **Decision Trees**

- The sequence of checking conditions and executing actions is critical → Trees explicitly show the order of decisions from left to right
- Trees naturally represent situations where certain conditions are only relevant down specific paths
- Avoids the clutter of listing irrelevant conditions like a table might